

Algorithm & Analysis

Sorting, Searching, Divide & Conquer, Analysis

Unit-03

Sorting

Unit-03.01

Why?

....an open-ended question

Preprocessing: Sorting is often a crucial step in preprocessing data before performing other operations. It helps in tasks like duplicate removal, data cleaning, and aggregation, which are common in data analysis and database management.

Algorithm Design and Analysis: Many algorithms and data structures rely on sorted data or perform better when the data is sorted. For example, various divide-and-conquer algorithms, such as merge sort and quicksort, heavily rely on sorting as a subproblem. Analyzing the time complexity and efficiency of algorithms often involves considering how they handle sorted or partially sorted data.

Data Presentation: Sorted data is easier for humans to interpret and work with. Whether it's a list of names, numbers, or any other type of data, sorting makes it easier to find specific items, identify patterns, and analyze trends.

Data Compression and Encoding: Sorted data can sometimes be compressed more effectively, leading to better compression ratios in data storage and transmission. Additionally, sorted data can simplify encoding schemes, reducing the overhead of representing the data.

Efficiency: Sorting allows for efficient searching, as sorted data can be easily searched using algorithms like binary search, which has a time complexity of $O(\log n)$. Unsorted data typically requires linear search, which has a time complexity of $O(n)$. In scenarios where there is a large volume of data, the efficiency gained from sorting becomes crucial.

Optimization: Sorting can often lead to optimization opportunities in various applications. For instance, in databases, sorted indexes can significantly improve query performance. In graphics rendering, sorting objects based on their depth can optimize rendering algorithms. Sorting also facilitates efficient merging and joining of datasets, which is essential in various data processing tasks.

Decision Making: Sorted data can facilitate decision-making processes by providing a clear hierarchy or order. For example, in financial markets, sorted data helps investors identify top performers or detect anomalies more efficiently.

2

1

4

1

3

9

Rules are simple

- You can view 2 box at max
- You can swap only two boxes at a time



SelectionSort

Unit-03.01.01

```
for (j = 0; j < n-1; j++)
```

```
    int iMin = j;
```

```
    for (i = j+1; i < n; i++)
```

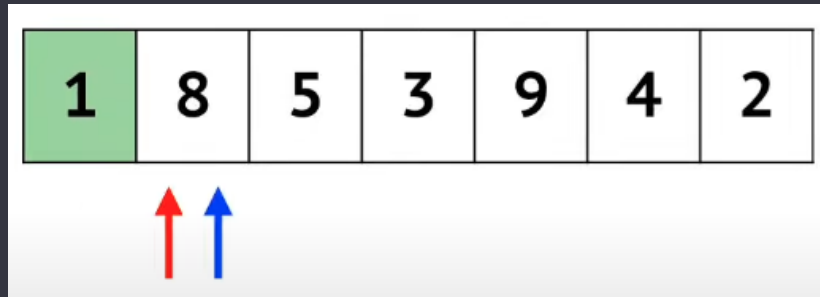
```
        if (a[i] < a[iMin])
```

```
            iMin = i;
```

```
    if (iMin != j)
```

```
        swap(a[j], a[iMin]);
```

Current Min  Current Item 



```
for (j = 0; j < n-1; j++)  
    int iMin = j;  
  
    for (i = j+1; i < n; i++)  
        if (a[i] < a[iMin])  
            iMin = i;  
  
    if (iMin != j)  
        swap(a[j], a[iMin]);
```

How many times this
would run?

Complexity Analysis

Unit-03.02

$$f(n) = \frac{n}{2} \times (n + 1) = \frac{n^2 + n}{2}$$

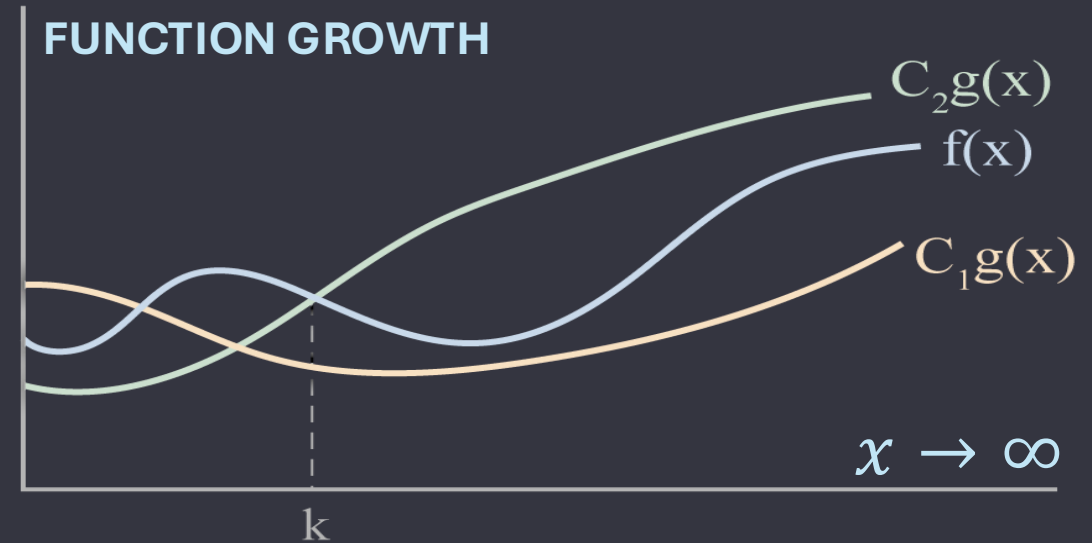
Complexity Analysis

Time Complexity: Time complexity measures the amount of time an algorithm takes to execute as a function of the size of the input. It provides an upper bound on the running time of an algorithm.

Space Complexity: Space complexity measures the amount of memory space an algorithm requires as a function of the input size.

Best, Worst, and Average Cases: Algorithms can have different time and space complexities depending on the characteristics of the input data.

Asymptotic Analysis: Complexity analysis typically focuses on the behavior of an algorithm as the input size approaches infinity. This is known as asymptotic analysis. It helps identify the dominant factors affecting the algorithm's performance and disregards constants and lower-order terms, focusing on the most significant aspects that determine scalability.



$$\begin{aligned} c_2g(x) &\geq f(x) \\ f(x) &= O(g(x)) \\ \text{for some } x &> k \end{aligned}$$

$$\begin{aligned} c_2g(x) &\geq f(x) \geq c_1g(x) \\ f(x) &= \Omega(g(x)) \\ \text{for some } x &> k \end{aligned}$$

$$\begin{aligned} c_2g(x) &\geq f(x) \geq c_1g(x) \\ f(x) &= \Theta(g(x)) \\ \text{for some } x &> k \end{aligned}$$

Comparing Algorithms: Complexity analysis allows for comparing different algorithms to determine which one is more efficient for a given problem. Algorithms with lower time or space complexity are generally preferred, but other factors such as ease of implementation and practical considerations may also influence the choice.

```
for (j = 0; j < n-1; j++)  
    int iMin = j;  
  
    for (i = j+1; i < n; i++)  
        if (a[i] < a[iMin])  
            iMin = i;  
  
    if (iMin != j)  
        swap(a[j], a[iMin]);
```

What
is
Big-Theta
here?

```
for (j = 0; j < n-1; j++)  
    int iMin = j;  
    for (i = j+1; i < n; i++)  
        if (a[i] < a[iMin])  
            iMin = i;  
  
    if (iMin != j)  
        swap(a[j], a[iMin]);
```

What
is
Big-Theta
here?

$$f(n) = \sum_{j=0}^{j < n-1} \sum_{i=j+1}^{j < n} 1$$

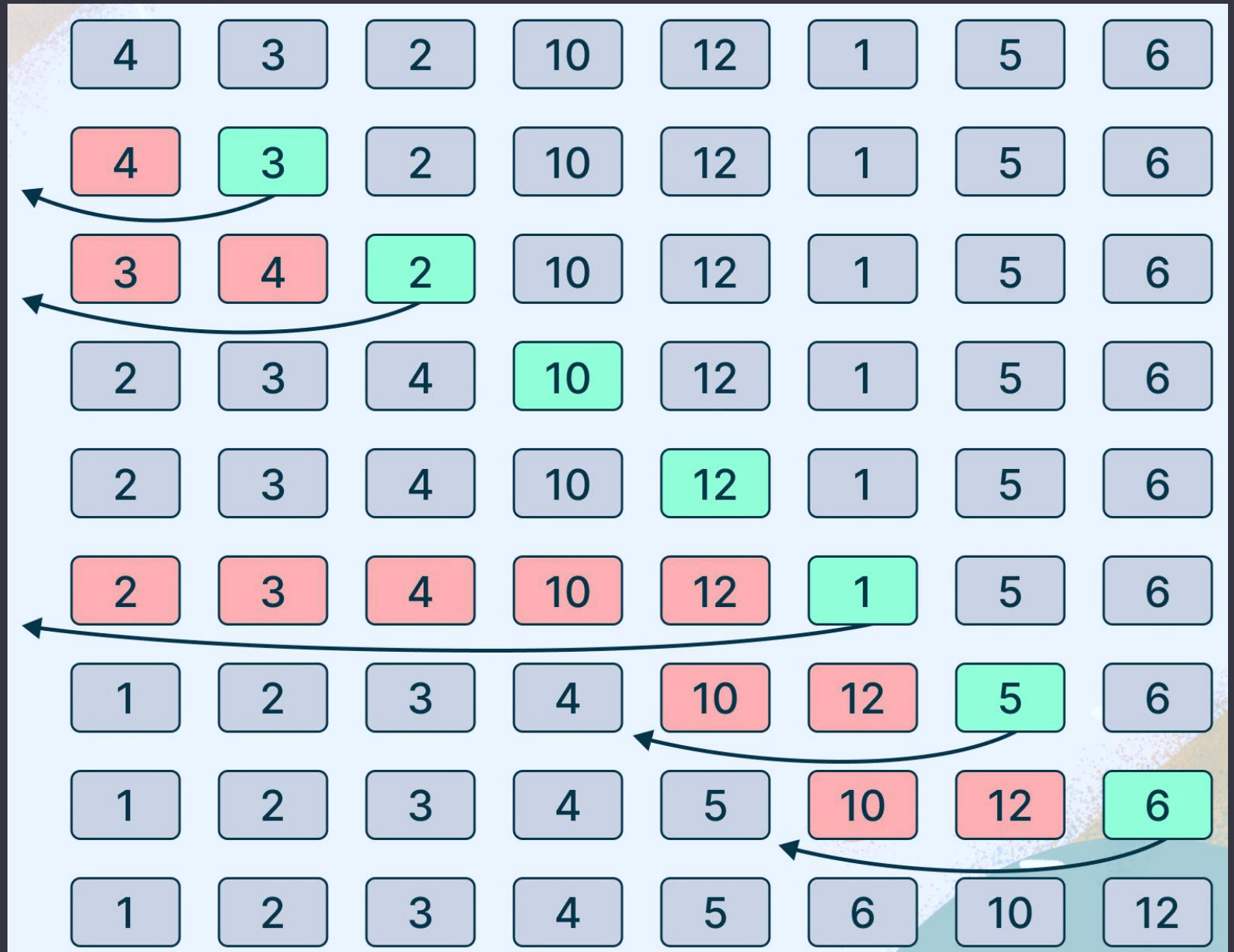
f(n) is $\Theta(n^2)$

HOW?

InsertionSort

Unit-03.01.02

```
for i : 1 to length(A) - 1
  j = i
  while j > 0 and A[j-1] > A[j]
    swap A[j] and A[j-1]
    j = j - 1
```



```

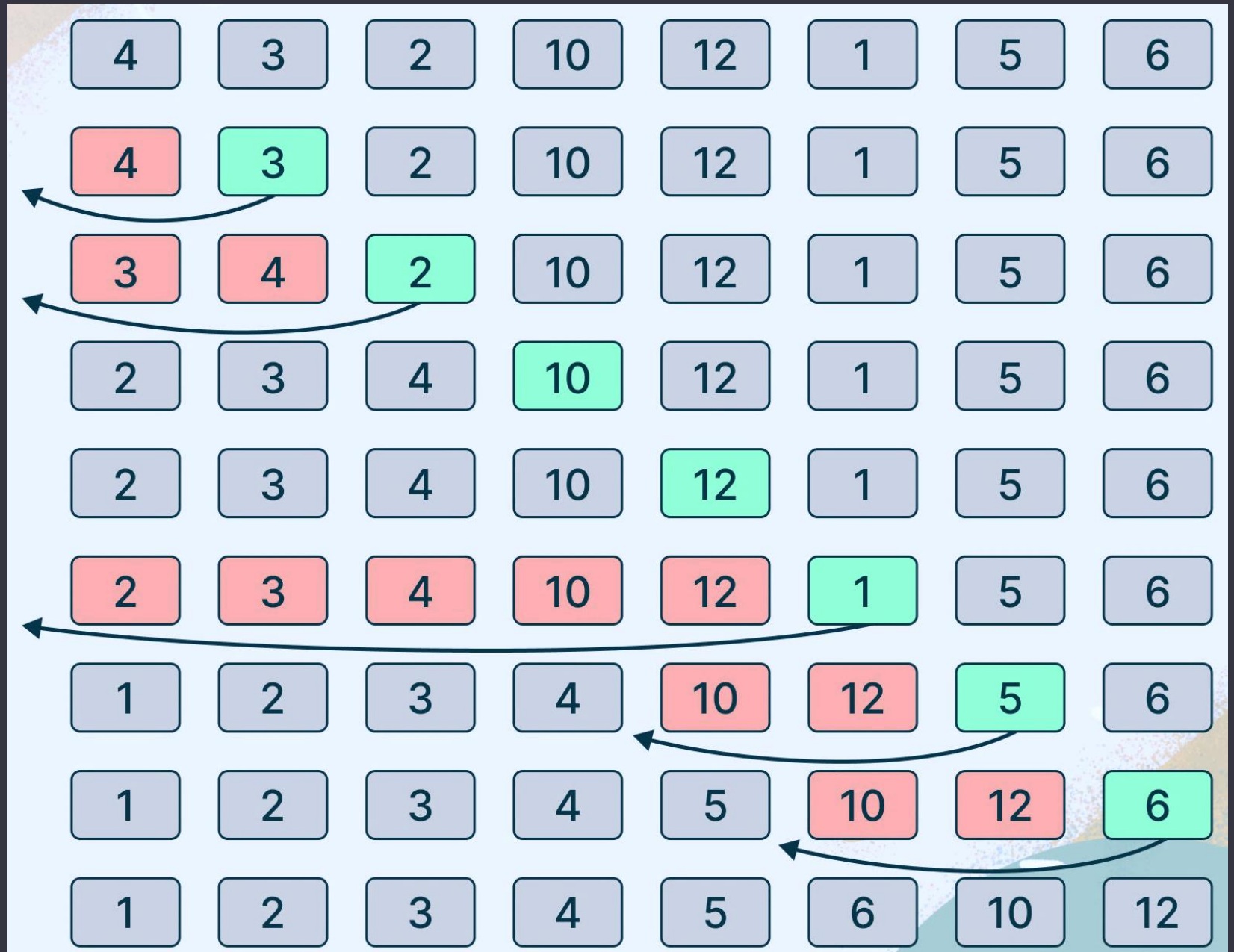
for i : 1 to length(A) - 1
  j = i
  while j > 0 and A[j-1] > A[j]
    swap A[j] and A[j-1]
    j = j - 1

```

Complexity
Analysis.
?? HOW ??

$f(n)$ is $O(n^2)$, best case is n

Can you guess best case?



BubbleSort

Unit-03.01.03

↓
[9, 2, 6, 15, 3,] [2, 6, 9, 3, 15,]

[2, 9, 6, 15, 3,]

[2, 9, 6, 15, 3,]

[2, 6, 9, 15, 3,]

[2, 6, 9, 15, 3,]

[2, 6, 9, 15, 3,]

[2, 6, 9, 3, 15,] [2, 6, 3, 9, 15,]



```
for i from 1 to N
  for j from 0 to N - 1
    if a[j] > a[j + 1]
      swap ( a[j] , a[j + 1] )
```

$O(n^2)$

[2, 3, 6, 9, 15,]

↓
[9, 2, 6, 15, 3,] [2, 6, 9, 3, 15,]

[2, 9, 6, 15, 3,]

[2, 9, 6, 15, 3,]

[2, 6, 9, 15, 3,]

[2, 6, 9, 15, 3,]

[2, 6, 9, 15, 3,]

[2, 6, 9, 3, 15,] [2, 6, 3, 9, 15,]



```
for i from 1 to N
  for j from 0 to N - 1
    if a[j] > a[j + 1]
      swap ( a[j] , a[j + 1] )
```

$O(n^2)$

[2, 3, 6, 9, 15,]

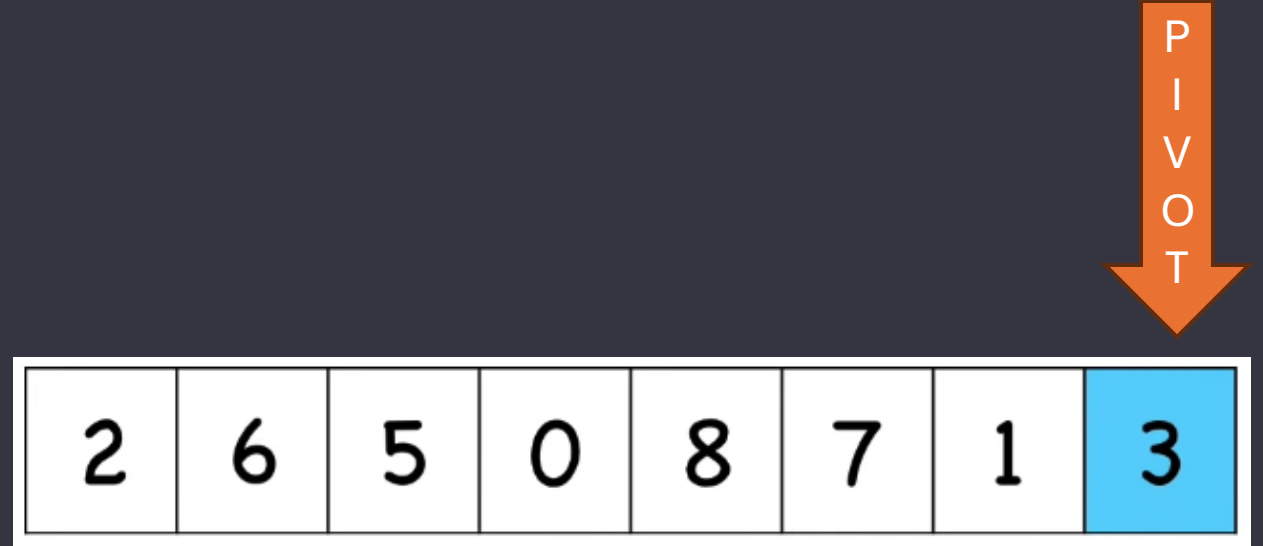
QuickSort

Unit-03.01.04

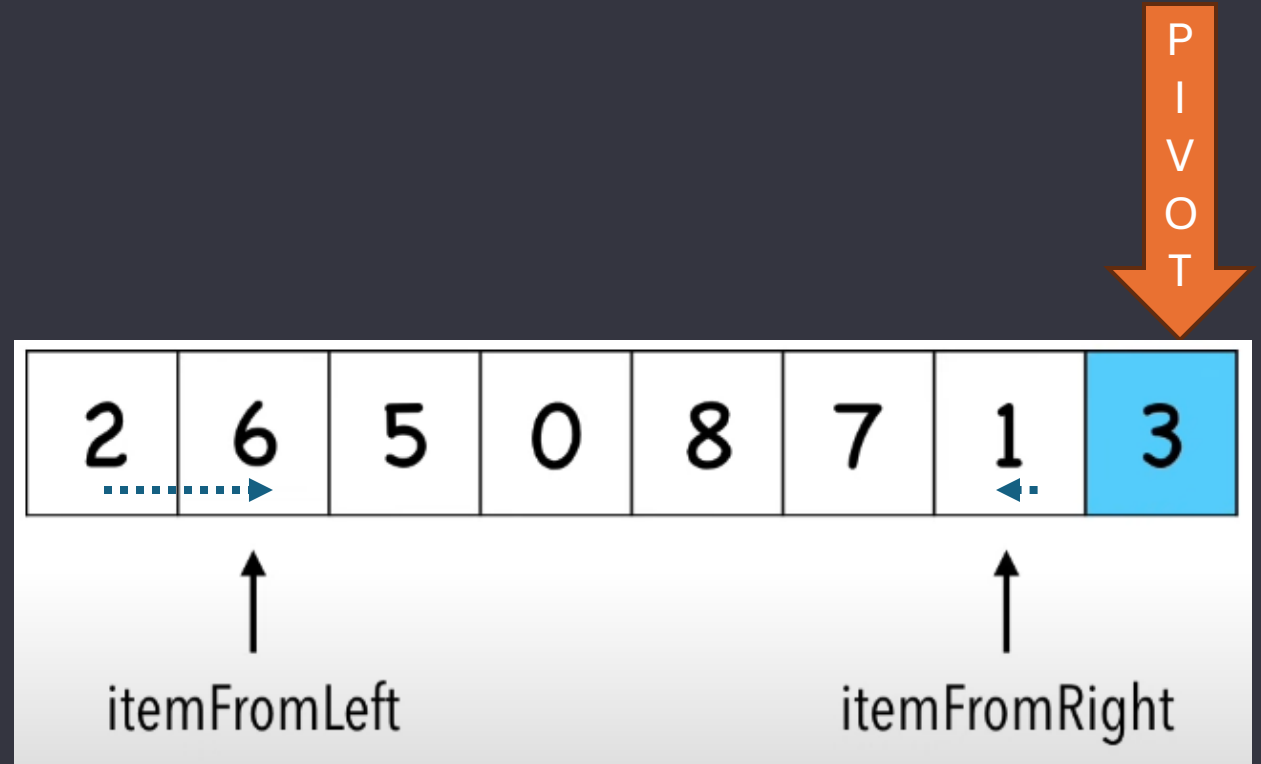
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger

2	6	5	0	8	7	1	3
---	---	---	---	---	---	---	---

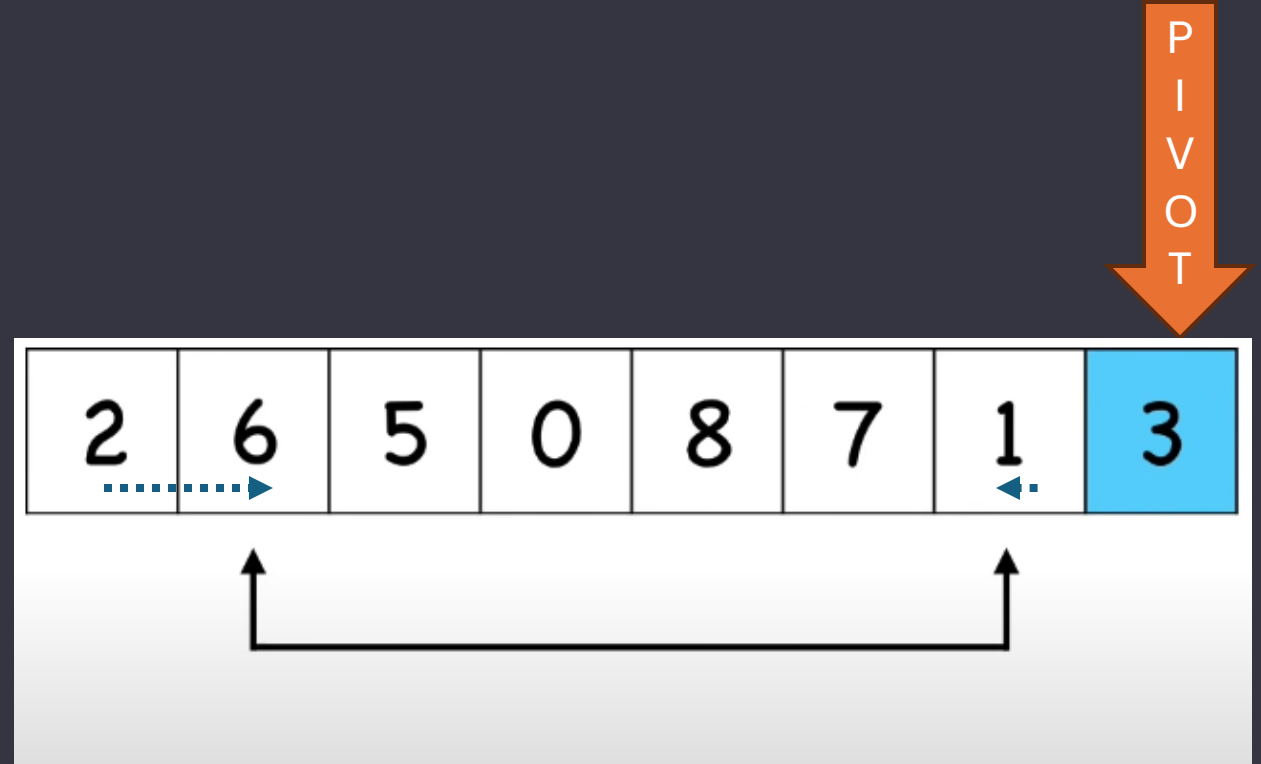
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



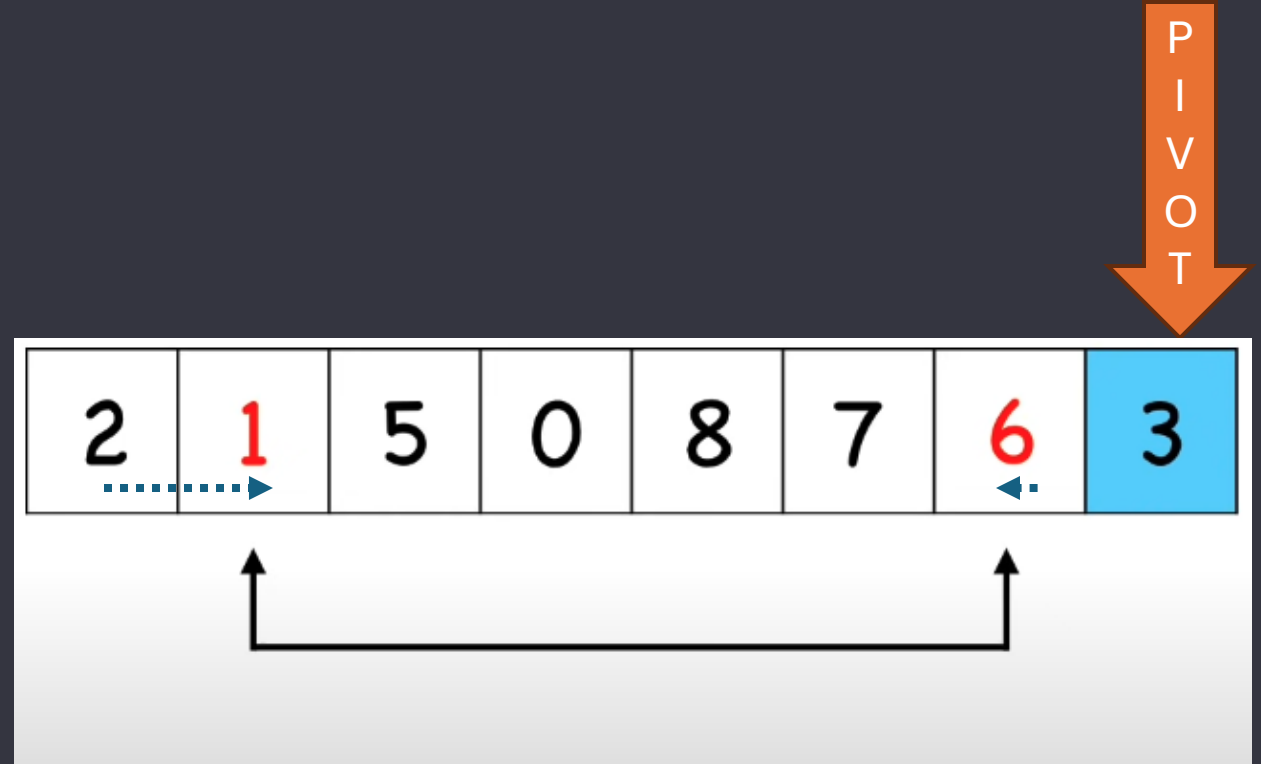
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



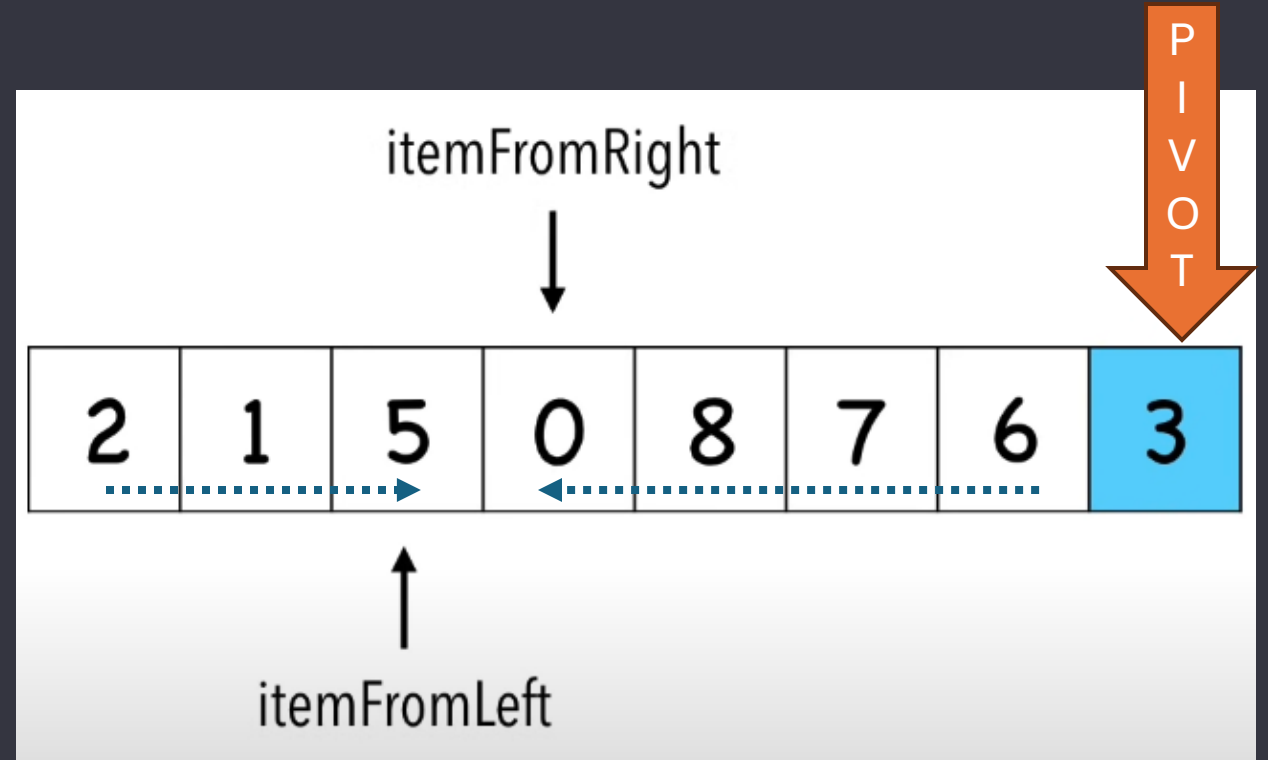
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



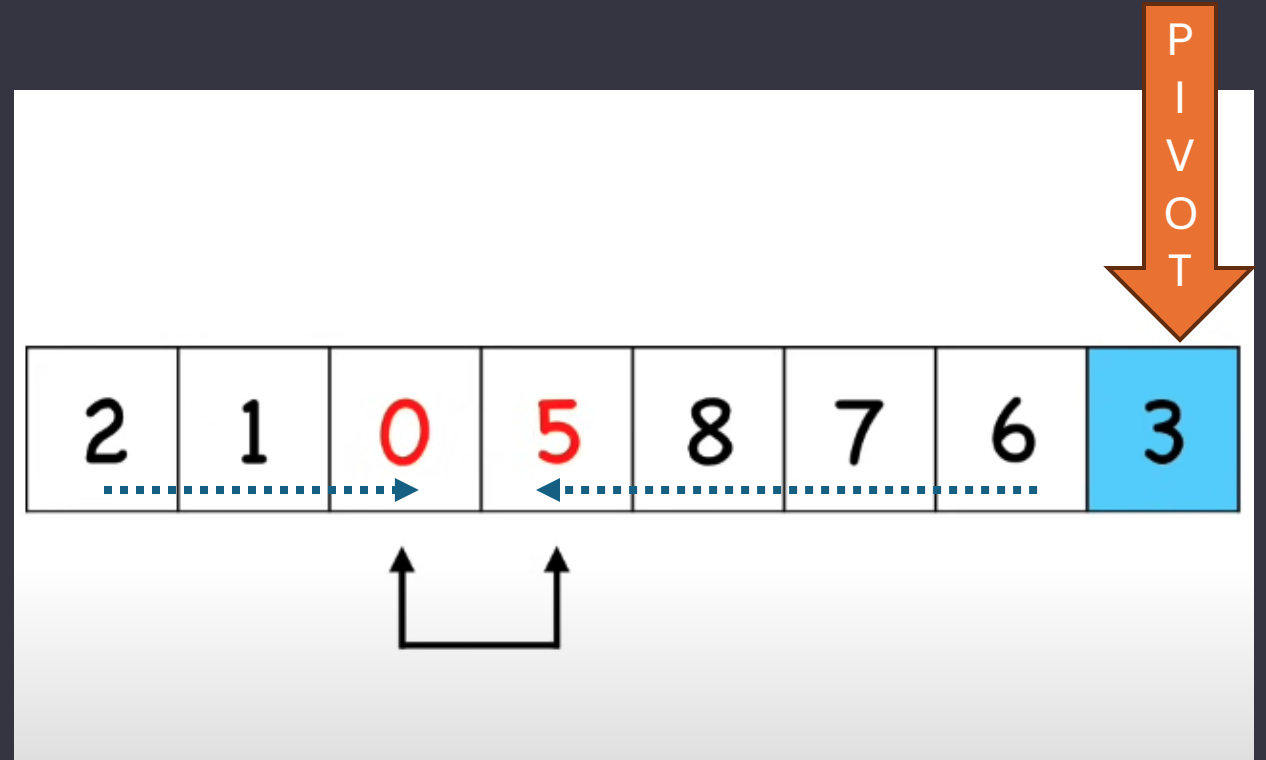
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



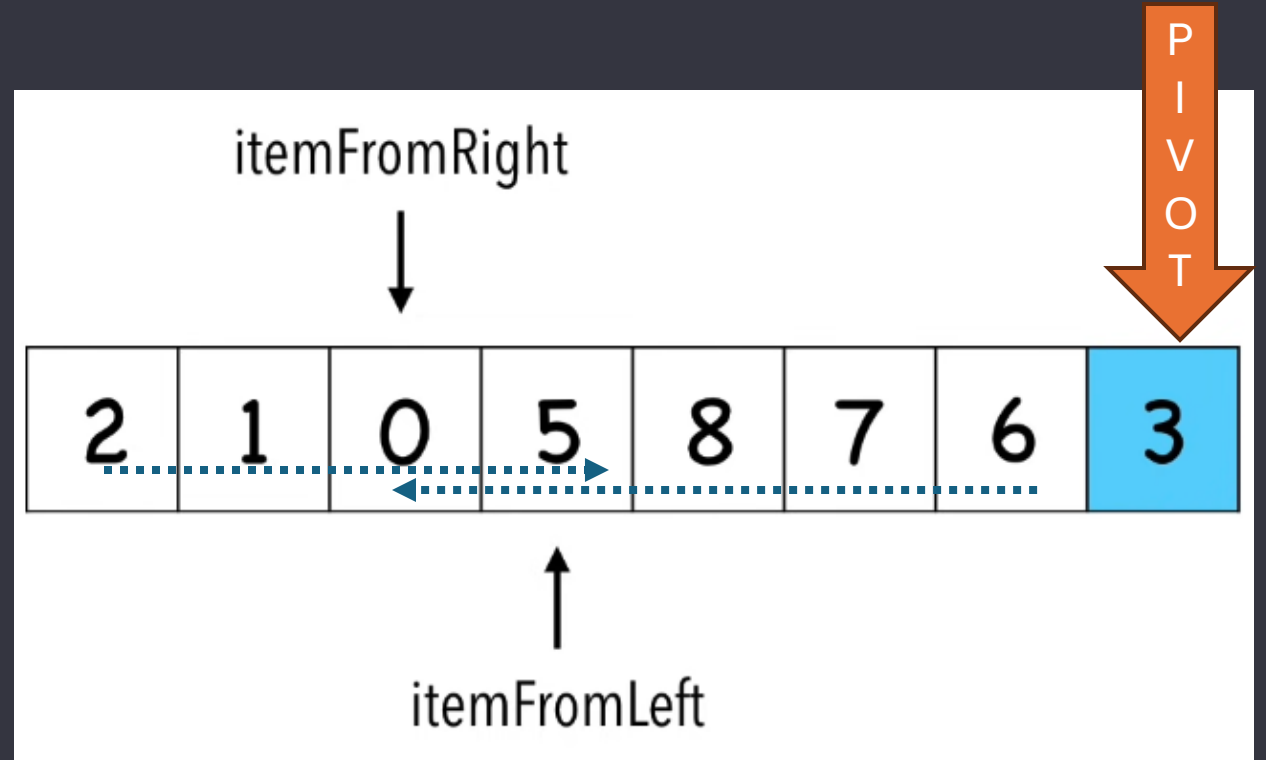
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



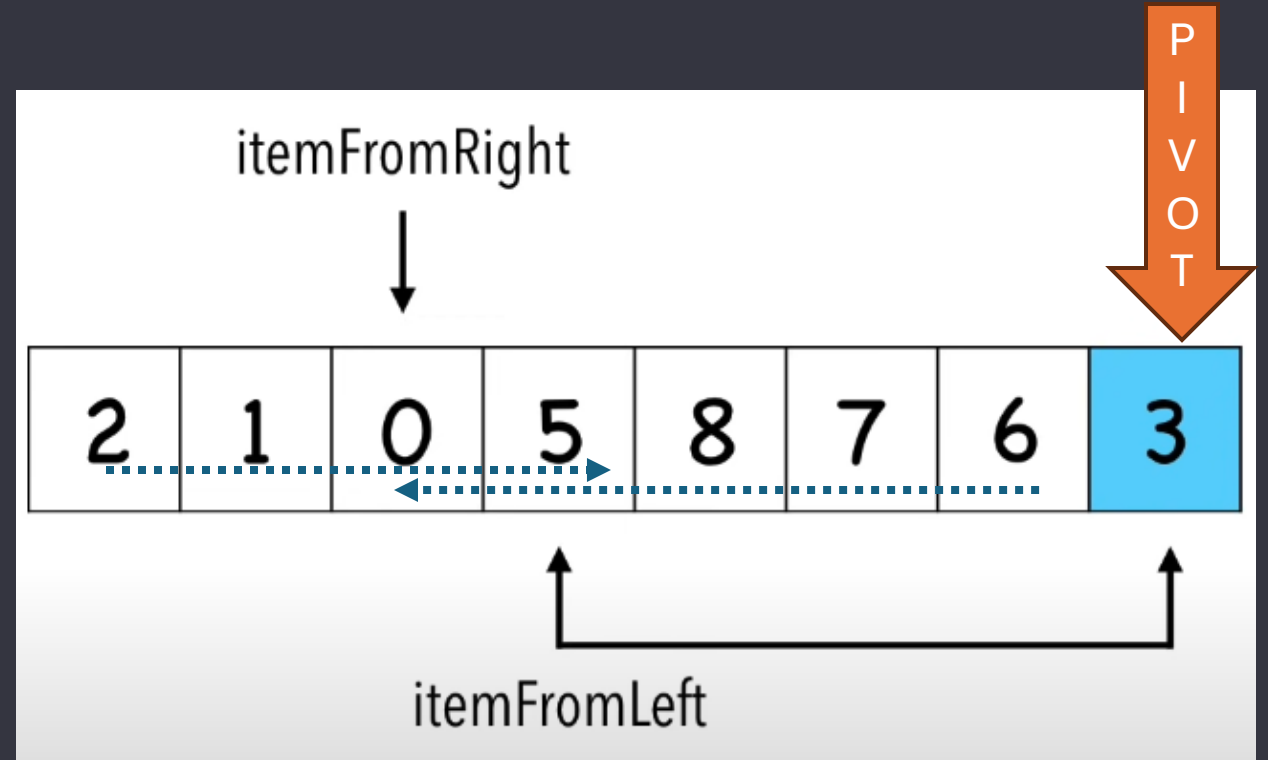
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



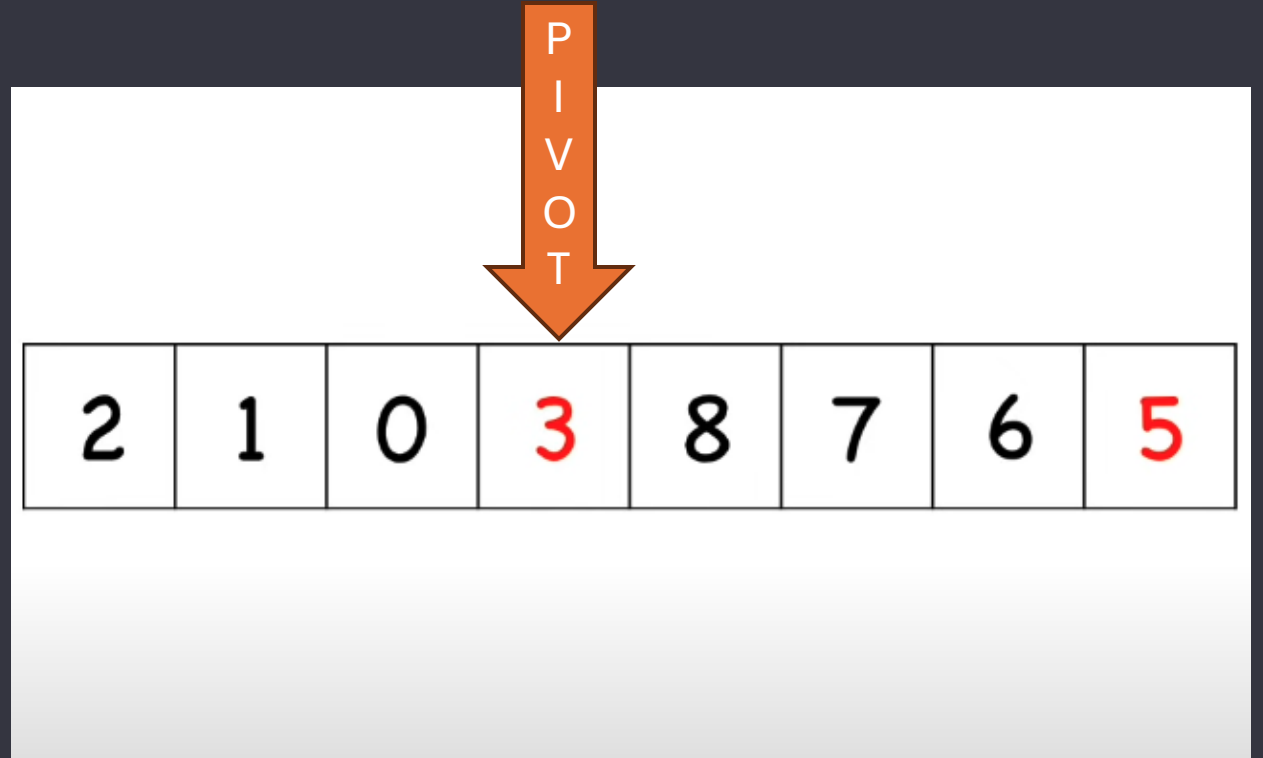
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



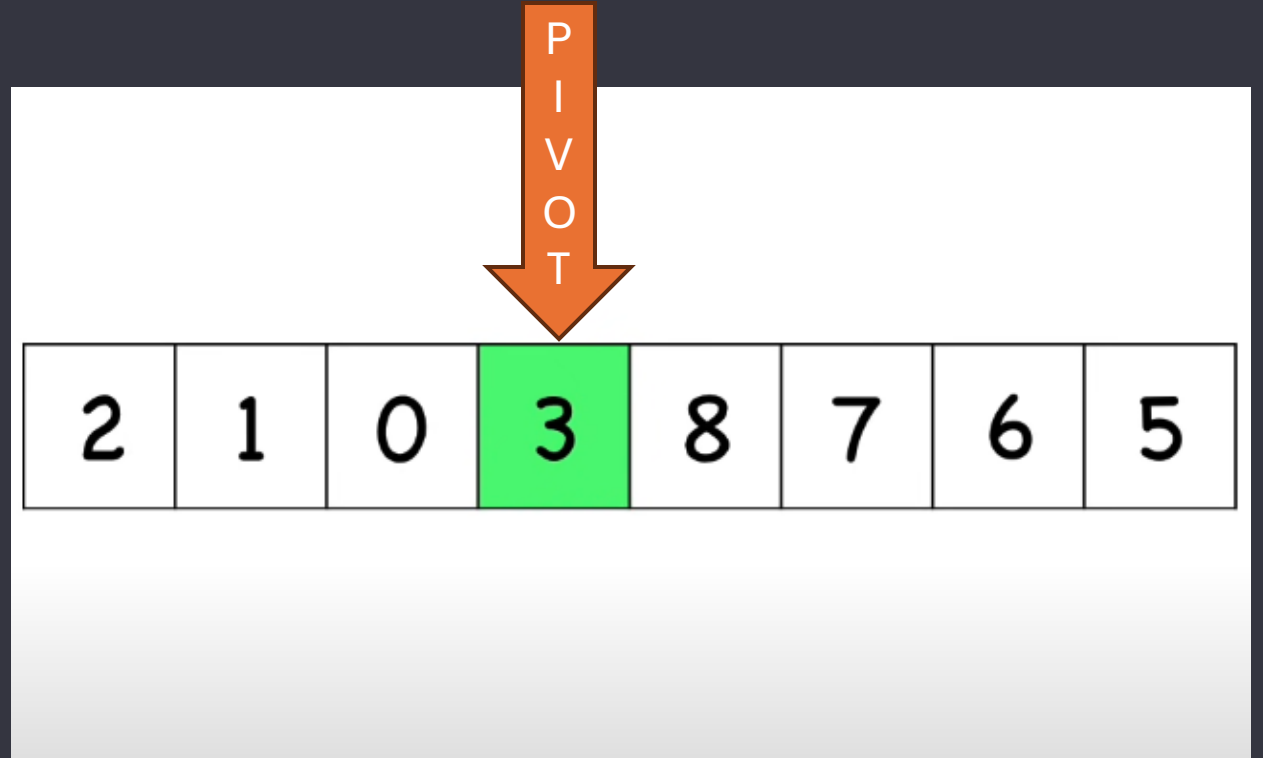
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



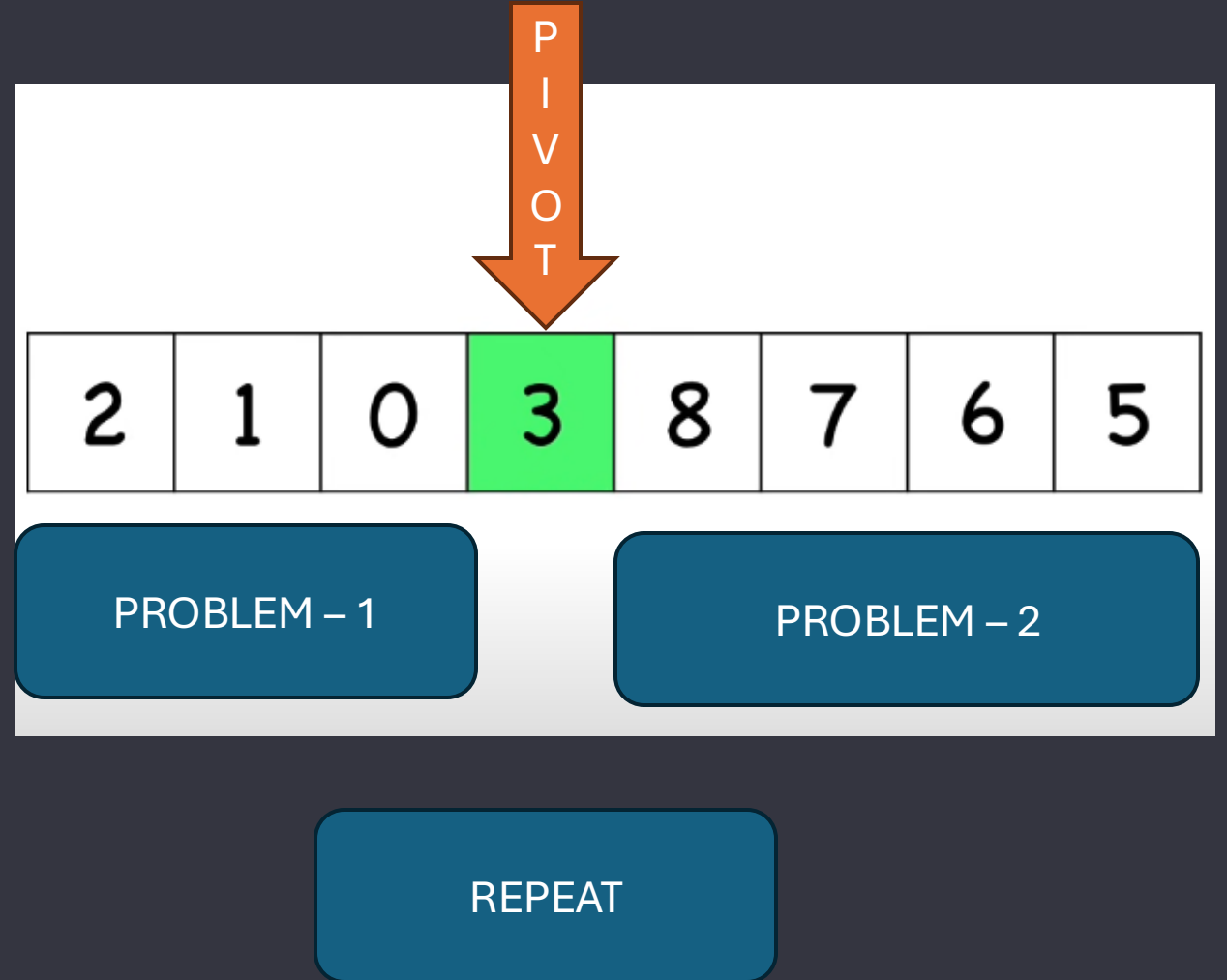
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



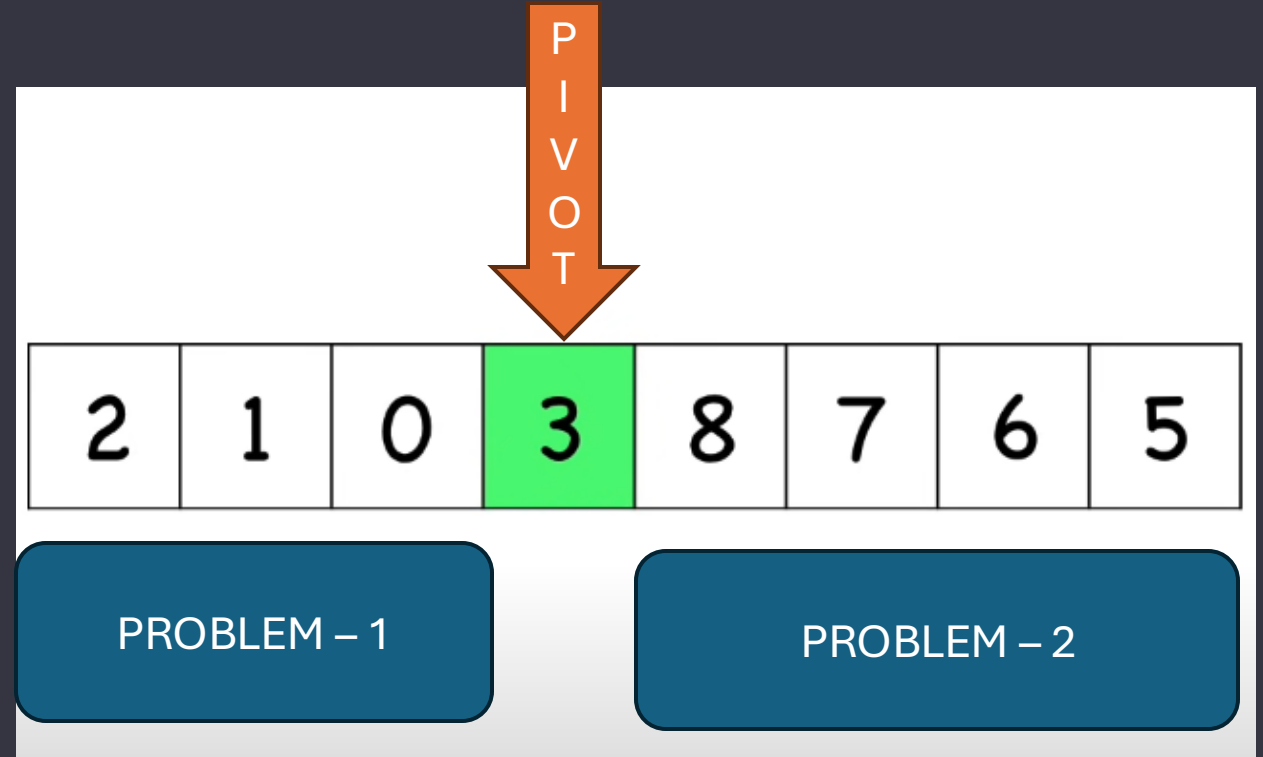
1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



1. Correct position in final, sorted array
2. Items to the left are smaller
3. Items to the right are larger



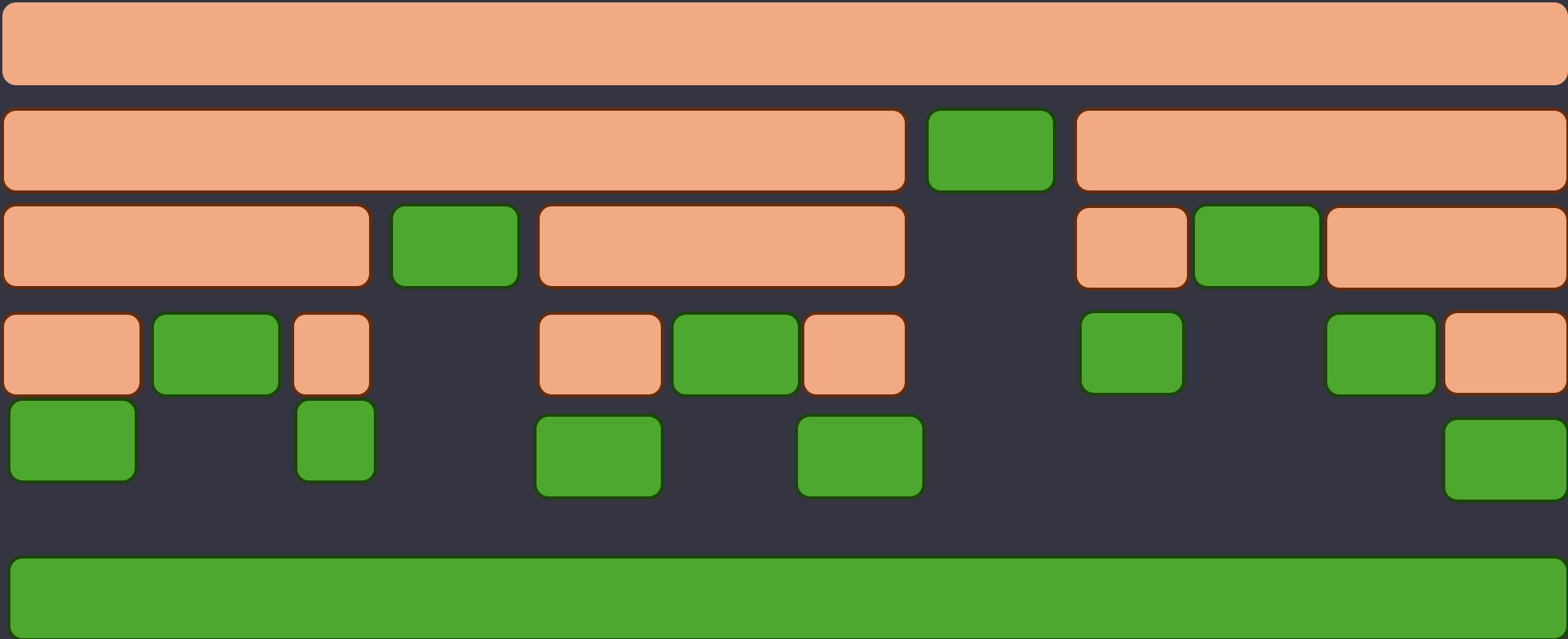
REPEAT

Resources: <https://www.youtube.com/watch?v=Hoixgm4-P4M>

Resources: <https://www.youtube.com/watch?v=WprjBK0p6rw&t=69s>

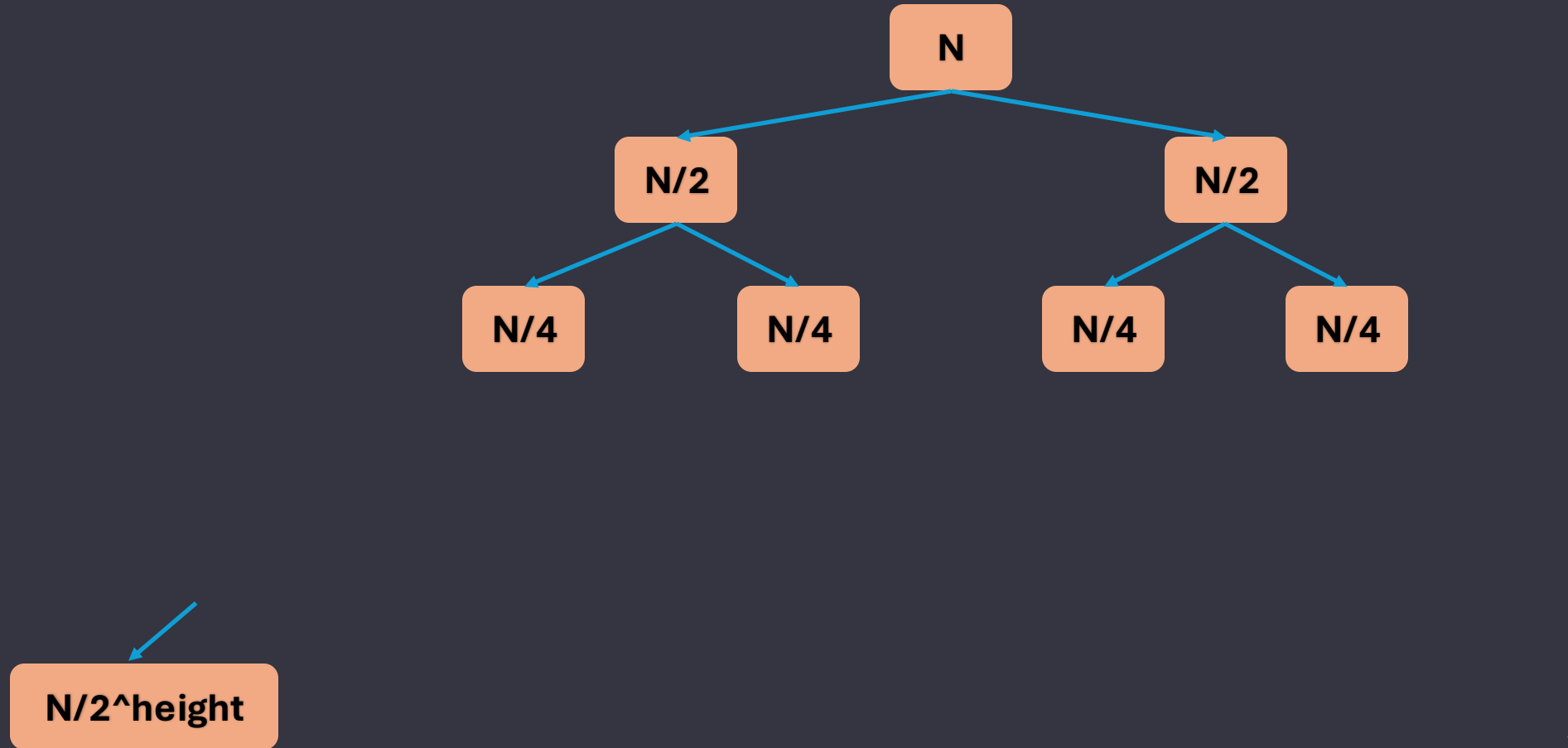
BUT HOW TO SOLVE?

QuickSort

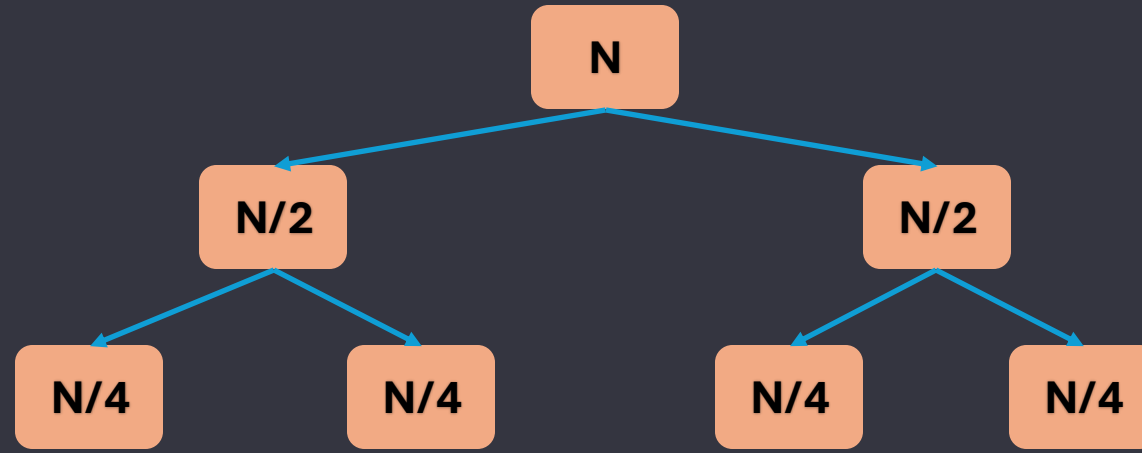


Algorithms | Divide & Conquer

Unit-03.03



How tall this tree will grow?

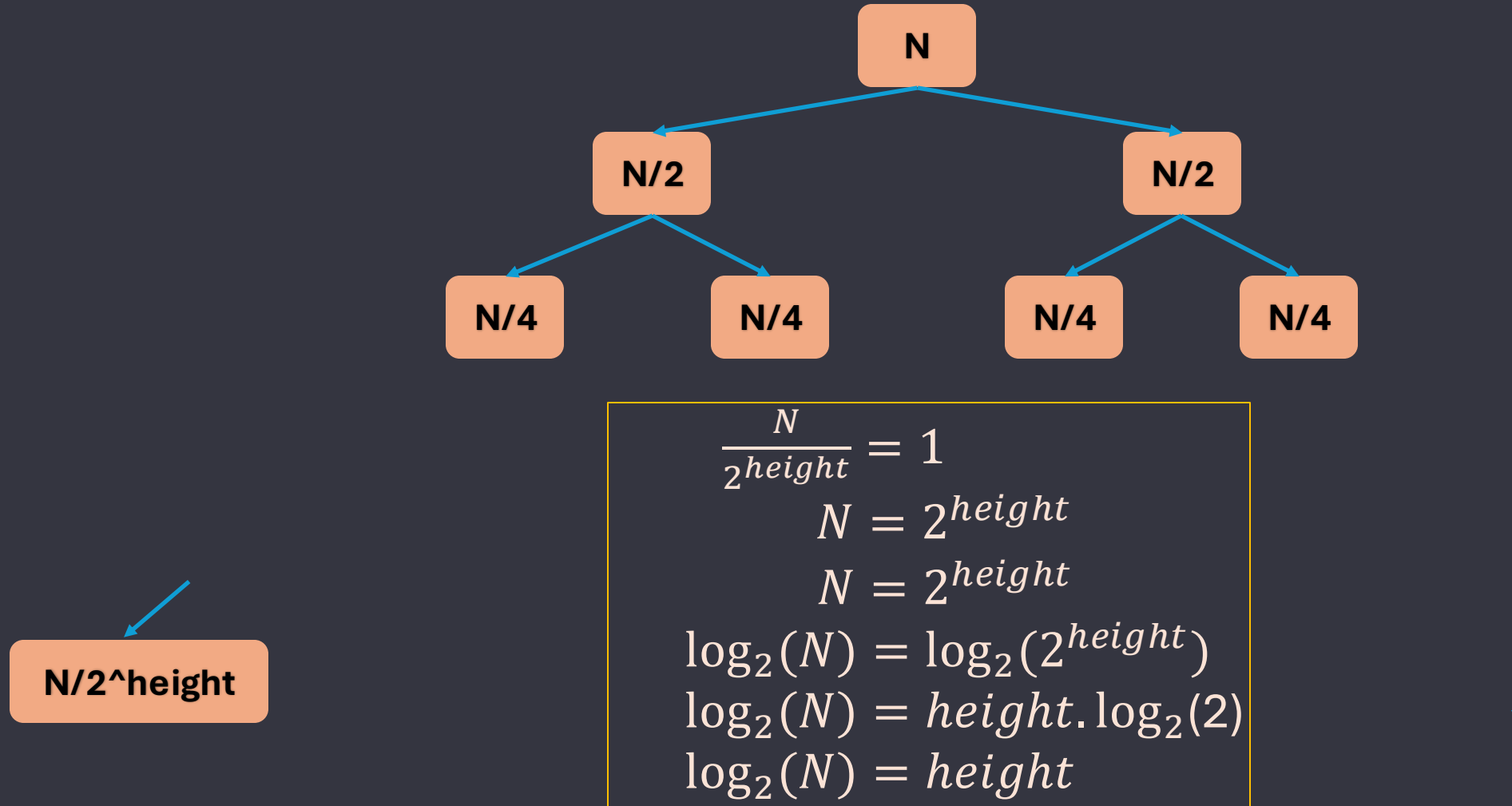


$N/2^{\text{height}}$

$$\frac{N}{2^{\text{height}}} = 1, \text{height} = ?$$

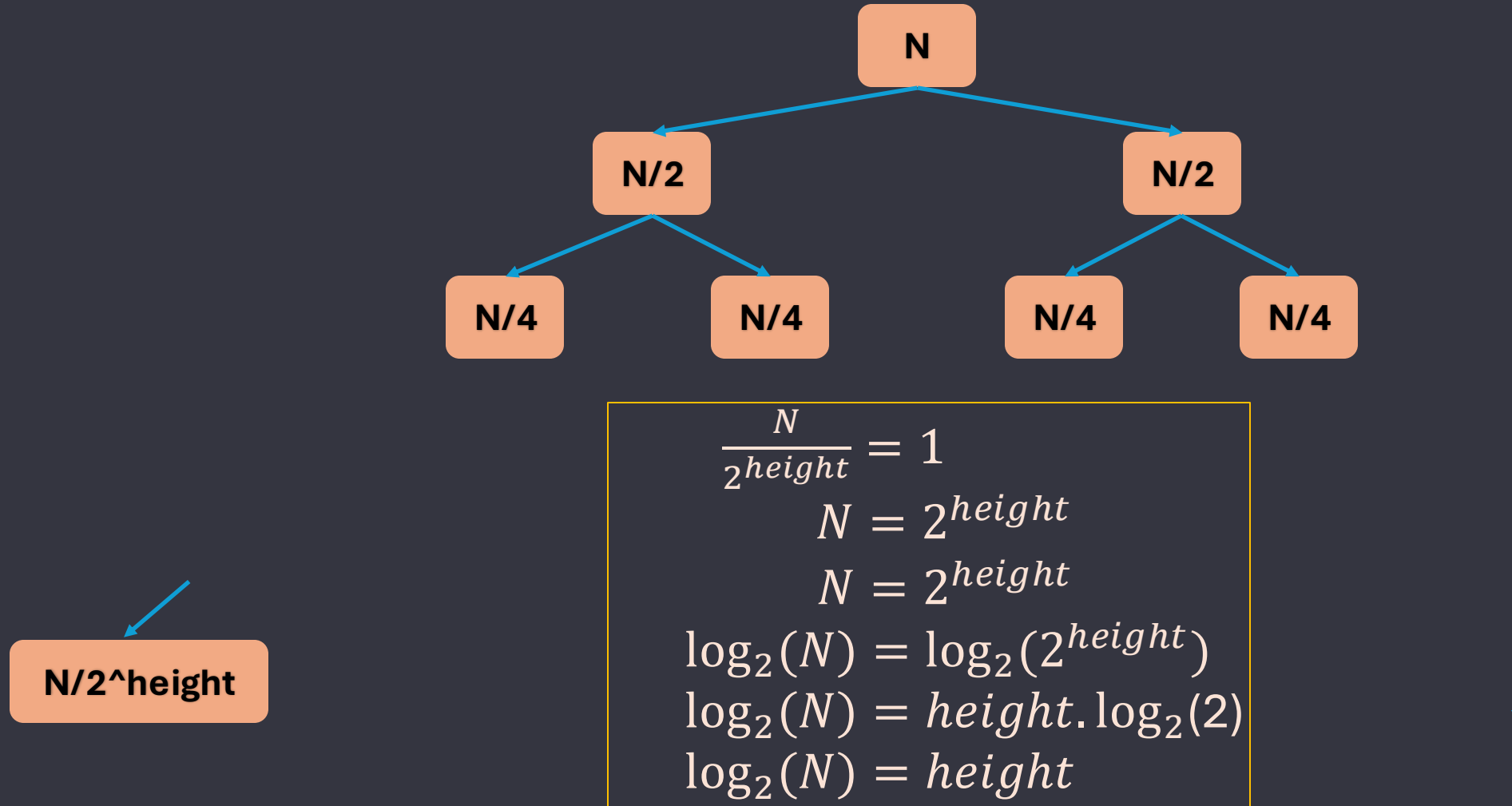
How tall this tree will grow?

Height is the number of steps we would need...



How tall this tree will grow?

N times Height is the number of steps we would need...



How tall this tree will grow?

N times Height is the number of steps we would need...

So total steps required are $N \log N$

VERIFY FOLLOWING.....bring anything concerning to discussion.

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$

Note:

- "n" refers to the number of elements in the array to be sorted.
- "k" refers to the range of input values.

Radix sort, not related with course content

3	5	2	1
011	101	010	001
011	101	010	001
010	011	101	001
101	001	010	011
001	010	011	101
1	2	3	5

Searching

Unit-03.04

```

#include <iostream>
#include <random>
#include <cmath>
using namespace std;

int randInt(int, int);

int LOW = 1, HIGH = 100;

int randInt(int low, int high) {
    random_device rd; // Obtain a random number from hardware
    mt19937 eng(rd()); // Seed the generator
    uniform_int_distribution<> distribution(low, high);
    return distribution(eng); // Generate the random number
}

```

```

int main(){
    play();
    return 0;
}

```

```

void play(){
    int num = randInt(1, 100), guess_num = -1, attempt = 0;
    while(true){
        attempt++;
        cout << "Enter your guess: ";
        cin >> guess_num;
        system("clear"); // Use system("clear"); windows user
        cout << "Enter your guess: " << guess_num;
        if(guess_num > num)
            cout << " !!! TOO HIGH " << endl;
        else if(guess_num < num)
            cout << " !!! TOO LOW" << endl;
        else
            break;
    }
    cout << endl << "Congratulations!!, You finished with "
        << attempt << " attempts." << endl;
}

```

Note, use cls for windows

```
#include <iostream>
#include <random>
#include <cmath>
using namespace std;

int randInt(int, int);

int LOW = 1, HIGH = 100;

int randInt(int low, int high) {
    random_device rd; // Obtain a random number from hardware
    mt19937 eng(rd()); // Seed the generator
    uniform_int_distribution<> distrib(low, high);
    return distrib(eng); // Generate the random number
}
```

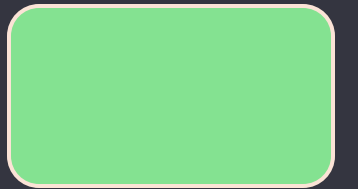
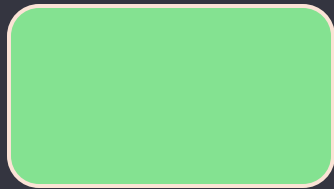
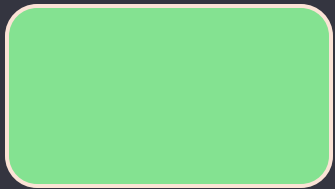
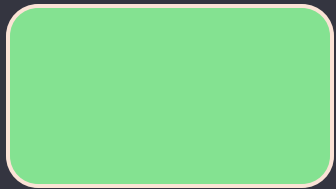
10M ?

```
int main(){
    play();
    return 0;
}
```

```
void play(){
    int num = randInt(1, 100), guess_num = -1, attempt = 0;
    while(true){
        attempt++;
        cout << "Enter your guess: ";
        cin >> guess_num;
        system("clear"); // Use system("clear"); windows user
        cout << "Enter your guess: " << guess_num;
        if(guess_num > num)
            cout << " !!! TOO HIGH " << endl;
        else if(guess_num < num)
            cout << " !!! TOO LOW" << endl;
        else
            break;
    }
    cout << endl << "Congratulations!!, You finished with "
        << attempt << " attempts." << endl;
}
```











1 - step

m - steps

n - step

**Worst Case = $O(n \log n) + O(n) \sim O(n \log n)$
Where is this $n \log n$ coming from?**



1 - step

m - steps

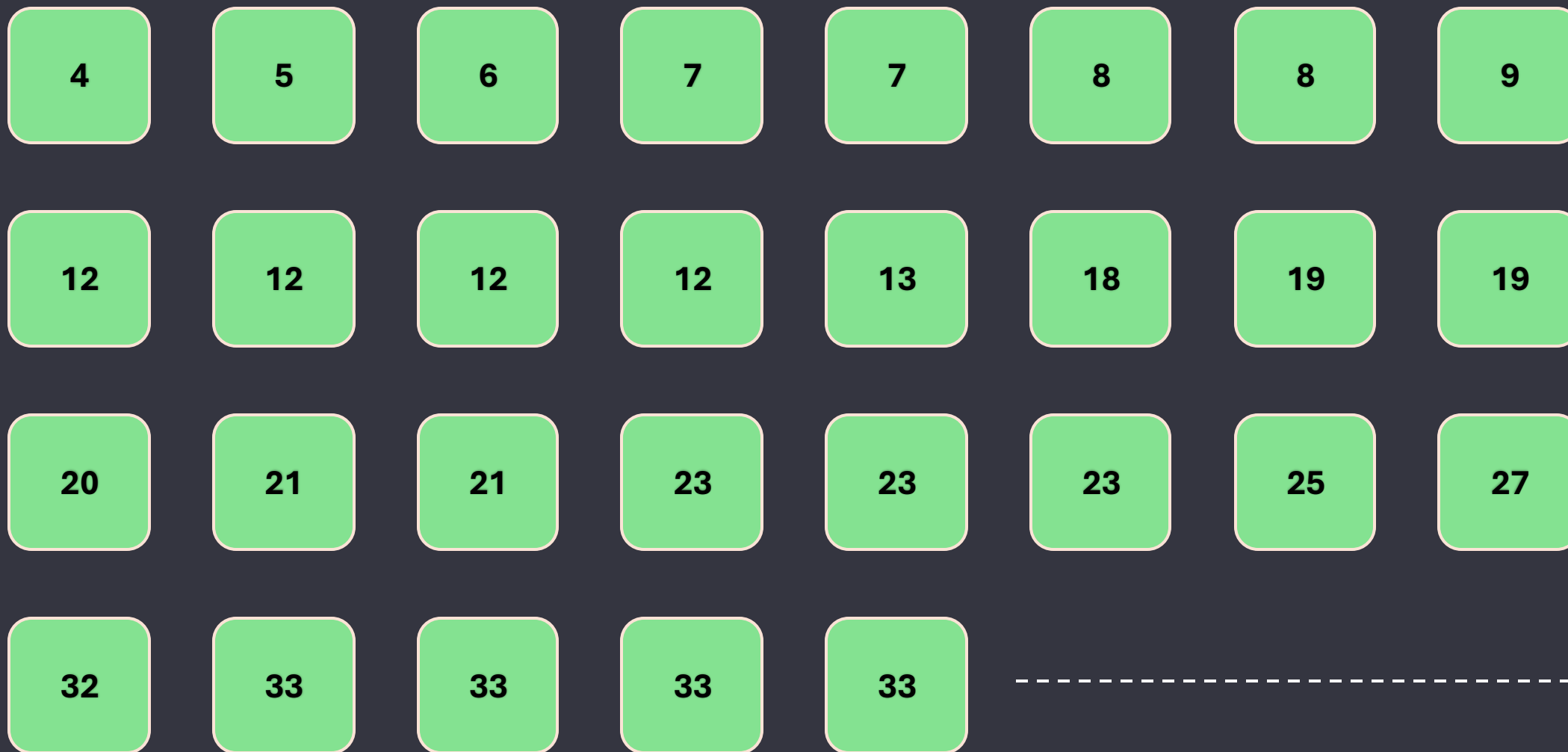
n - step

**Worst Case = $O(n \log n) + O(n) \sim O(n \log n)$
Where is this $n \log n$ coming from?**

$n \log n$ time for sorting the array

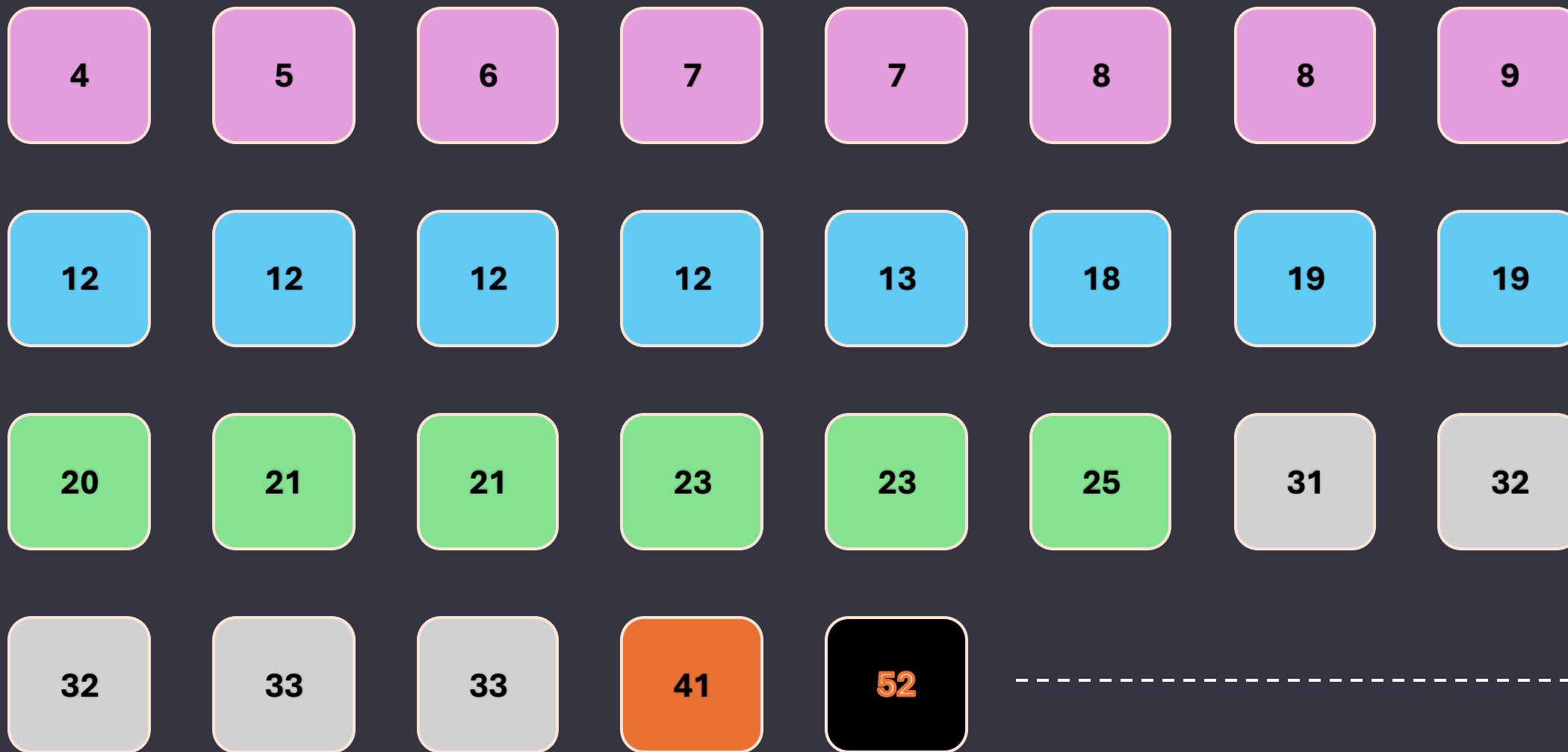
Can we improve?
(Let's think linearly one more time)

5	20	7	30	10	9	8	72
51	42	43	55	48	47	55	65
79	61	62	73	55	12	17	19
83	44	55	56	57	-----		



(*Assume different list sorted, compared to before)

4	5	6	7	7	8	8	9
12	12	12	12	13	18	19	19
20	21	21	23	23	25	31	32
32	33	33	41	52	-----		

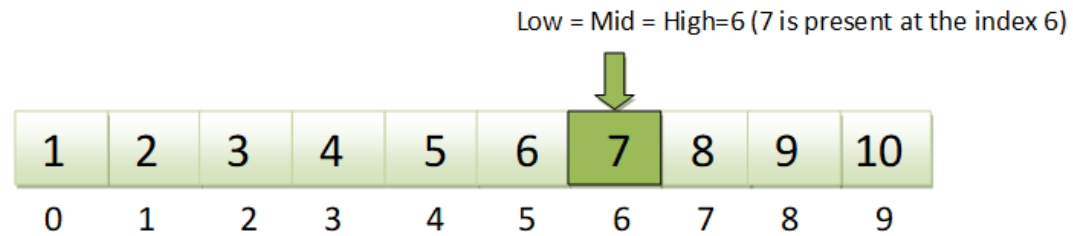
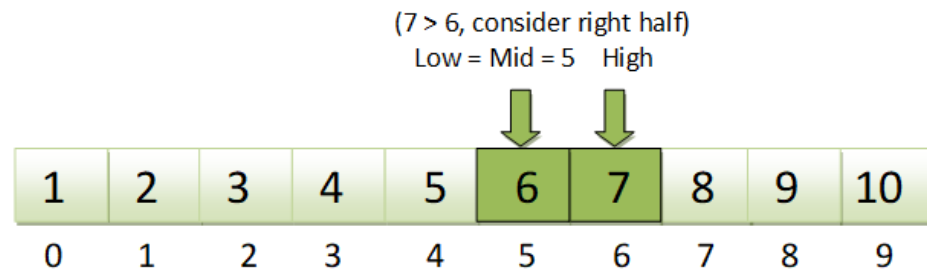
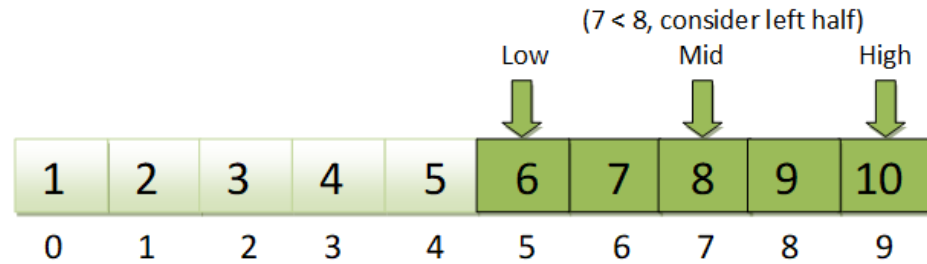
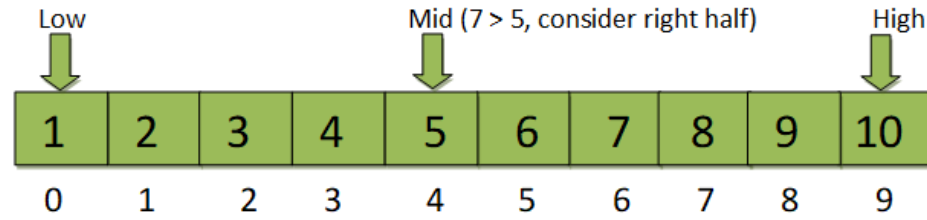


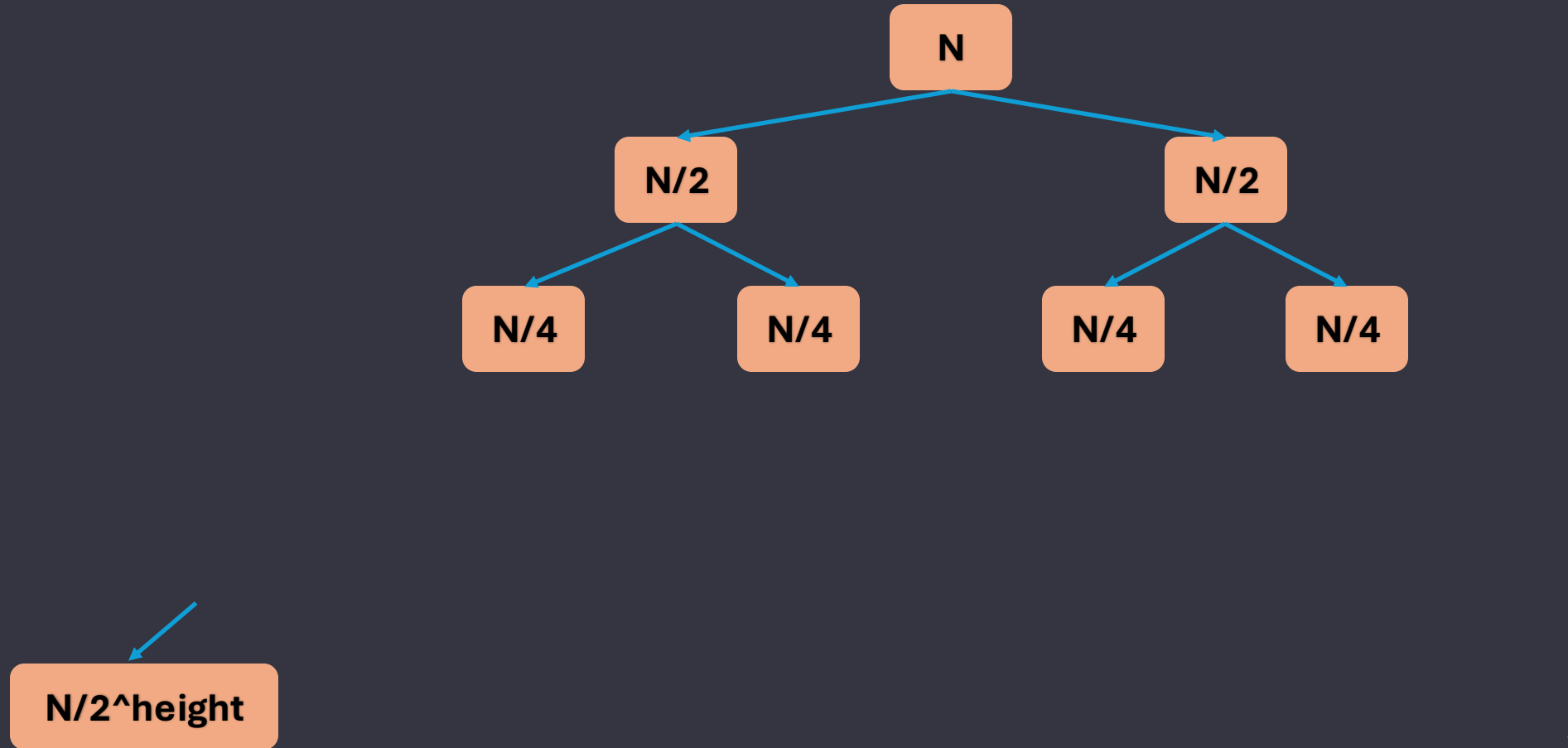
BUCKETIZATION: Bucket Search
COMPLEXITY?????

Linear Search is not efficient.
You need to visit every elements in worst case.

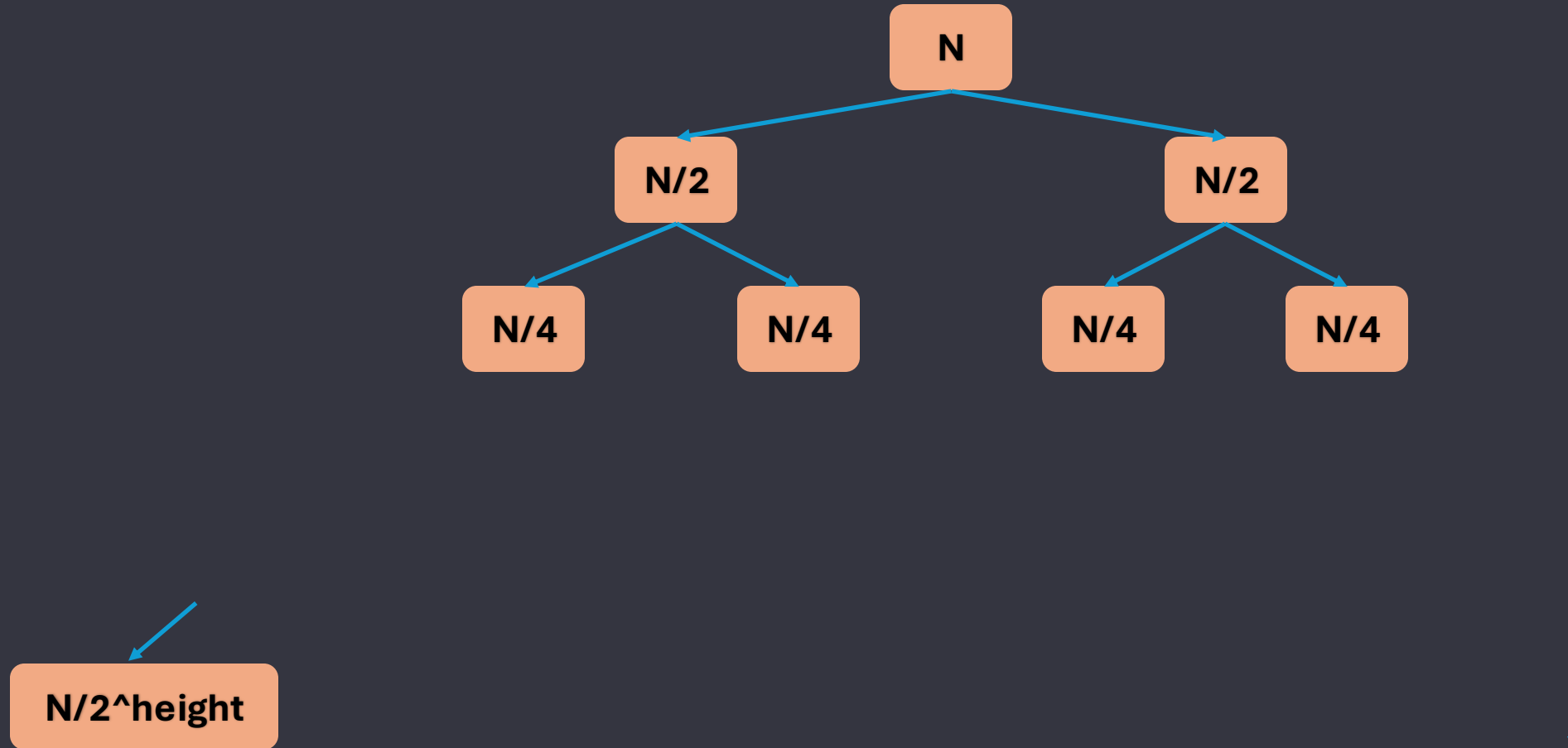
Binary Search

Search the number 7 in the array





How tall this tree will grow? What is our BigO



How tall this tree will grow? What is our BigO? $O(\log n)$



meet our Bobcat.....



We plan to invite 5 of them..... but there is a problem.....



they fight if we put them too close.....

0

2

7

10

18

22



Can we
place them
by... distance?

0 2 7 10 18 22



1 Unit



2 Unit



3 Unit



4 Unit



Can we
place them
by... distance?

0	2	7	10		18	22
						

1 Unit

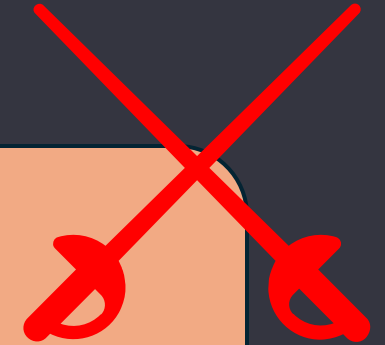
				
---	---	--	---	---

2 Unit

				
---	--	---	---	---

3 Unit

				
--	---	--	--	--



`arr [] = [ 0, 3, 4, 7, 9, 10], bobcats = 4`

What is the maximum minimum distance you can place 4 bobcats together given above placements?

arr [] = [0, 3, 4, 7, 9,99....845.....1M.....100M...2B]

4 bobcats, what is that minimum?

What is the maximum minimum distance you can place 4 bobcats together given above placements?